Multi Environment Application Deployment

Deployment guide and screenshot evidence

Student: Sudhanshu Joshi

Assessment date: 18 October 2026

Deployment target: Local Windows computer with Docker Desktop

Status: Working local deployment with a logged worker restart

Repository: https://github.com/sjoshigit/multienv-deployment-sudhanshu-joshi

Deployment outcome

The application was deployed locally using Docker Compose. Supplied screenshots show four running containers, a healthy MongoDB service, a working React dashboard, separate development and production tickets, successful API responses, and successful database pings from both Flask backends.

This report documents the configuration and commands used during deployment and includes all 24 supplied screenshots. Screenshots are preserved as evidence of the observed state, including the initial frontend build failure and a later Gunicorn worker restart. Repository publication and submission completion are not established by these screenshots.

Infrastructure and application design

Service	Runtime	Host to container port
frontend	React build served by Nginx	127.0.0.1:3000 → 80
backend-dev	Python 3.11 and Flask CLI	127.0.0.1:3001 → 3001
backend-prod	Python 3.11 and Gunicorn	127.0.0.1:3002 → 3002
mongo	MongoDB 7	27017 internally only

Local deployment was selected as permitted by the assessment. EC2 instances and AWS security groups are not applicable. The Windows host runs Windows 10 Enterprise Evaluation version 10.0.19045, 64-bit, with an Intel Core i5-11400H processor (6 cores and 12 logical processors) and 23.74 GiB reported memory. Docker Desktop 4.95.0 runs Linux containers with Docker client and engine 29.8.2 and Compose v5.6.0. Figures 20 to 24 provide the infrastructure evidence.

Compose provides a shared network and service-name DNS. Backends connect to mongo:27017. The browser calls the published backend ports through localhost. A named mongo_data volume stores database data independently of container lifetimes.

Docker and environment configuration

Each service has its own Dockerfile. Both backend images install requirements before copying source code to reuse cached dependency layers. The production image also installs Gunicorn and runs two workers. The frontend uses Node 22 Alpine for compilation and Nginx Alpine for static hosting.

File	Purpose
backend/dev/Dockerfile	Runs Flask on 0.0.0.0:3001
backend/prod/Dockerfile	Runs Gunicorn on 0.0.0.0:3002 with access and error logs
frontend/Dockerfile	Builds React and copies build output into Nginx
frontend/nginx.conf	Falls back to index.html for React routes
docker-compose.yml	Defines services, ports, env files, dependencies and volume
backend/dev/.env	Selects tickets_dev database
backend/prod/.env	Selects tickets_prod database

Environment values

backend/dev/.env
```
MONGO_URI=mongodb://mongo:27017/tickets_dev
```

backend/prod/.env
```
MONGO_URI=mongodb://mongo:27017/tickets_prod
```

Compose injects these values using env_file. Local .env files are excluded from version control and copied from .env.example for a fresh checkout. The supplied Flask source uses fixed backend ports. No frontend environment file is needed for its existing localhost API URLs.

Startup and routing

MongoDB uses a mongosh ping health check. Both backends wait for the database to become healthy. Frontend startup depends on backend startup, but this dependency alone is not an API readiness test. Connectivity is verified separately.

```
location / {
  try_files $uri $uri/ /index.html;
}
```

The Nginx fallback allows direct access to /dev and /prod. The React application fetches /api/tickets on port 3001 or 3002. Separate database names keep development and production ticket records apart.

Local security scope

Published ports bind to 127.0.0.1, and MongoDB has no published host port. This is a local demonstration: MongoDB authentication, HTTPS and application authentication are not configured. Gunicorn is used for production serving, but that alone does not make the application ready for public deployment.

Deployment procedure

Install Git and Docker Desktop on Windows, enable the Linux container engine and wait for Docker Desktop to report that the engine is running. Internet access is required for the first image build and package installation. Use PowerShell in the project folder.

```
docker version
docker compose version
git clone https://github.com/sjoshigit/multienv-deployment-sudhanshu-joshi.git
cd multienv-deployment-sudhanshu-joshi
```

Confirm that all three Dockerfiles and frontend/nginx.conf are present. If example environment files are included in the checkout, create the local configuration:

```
Copy-Item backend/dev/.env.example backend/dev/.env
Copy-Item backend/prod/.env.example backend/prod/.env
```

If examples are absent, create them using the MongoDB values on the preceding page. Validate, build and start the services:

```
docker compose config --quiet
docker compose --progress plain build
docker compose up -d
docker compose ps
```

Expected result: all four services are Up and MongoDB is healthy. Screenshot 460 shows network and volume creation, service startup, and published port mappings.

Application access

Purpose	URL
Dashboard	http://localhost:3000
Development frontend	http://localhost:3000/dev
Production frontend	http://localhost:3000/prod
Development ticket management	http://localhost:3001
Production ticket management	http://localhost:3002
Development API	http://localhost:3001/api/tickets
Production API	http://localhost:3002/api/tickets

The assessment dashboard URL ending in :300 is interpreted as a typo; the specified frontend port is 3000. These addresses work on the deployment computer while the services are running.

Testing and verification

Test	Observed result	Screenshot
Container status	Four services Up; MongoDB healthy	460 and 461
Dashboard	Environment selection page loads	462
Production ticket	production test is created and displayed	467 and 468
Development ticket	dev test is created and displayed	469 and 470
React integration	Each environment displays its matching ticket	471 and 472
JSON APIs	Separate environment labels and task data returned	473
Database connectivity	Both backend pings return ok 1.0	474
Application logging	Successful HTTP requests; worker restart also present	475

Repeatable test commands

```
Invoke-RestMethod http://localhost:3001/api/tickets |
  ConvertTo-Json -Depth 5
Invoke-RestMethod http://localhost:3002/api/tickets |
  ConvertTo-Json -Depth 5
```

```
docker compose exec backend-dev python -c "from app import mongo;
print(mongo.cx.admin.command('ping'))"
docker compose exec backend-prod python -c "from app import mongo;
print(mongo.cx.admin.command('ping'))"
```

For the two ping commands above, keep the Python code inside each pair of double quotes on one line when entering it in PowerShell.

```
docker compose logs --tail=50
docker stats --no-stream
```

Browser test: create a different ticket on each Flask page, refresh the matching React page, and confirm that development and production lists remain distinct. These tests cover creation, retrieval and database connectivity. Screenshots do not demonstrate completion/deletion actions, persistence after restart, or sustained load testing.

Stop and restart

```
docker compose down
docker compose up -d
```

The named volume remains after docker compose down. Adding -v deletes the volume and its stored tickets; it is not part of the normal stop procedure.

Troubleshooting and remaining limitations

Missing Dockerfiles

The first build could not find Dockerfiles. The fix was to create backend/dev/Dockerfile, backend/prod/Dockerfile and frontend/Dockerfile with the exact filename and matching Compose build contexts.

Docker Desktop connectivity

Earlier terminal errors indicated an unavailable Linux engine and a named-pipe access conflict. Troubleshooting included updating the client, restarting Docker Desktop and checking docker version for both Client and Server sections. Subsequent screenshots establish successful engine operation; they do not identify the precise cause of the earlier access conflict.

Frontend compilation failure

Screenshots 455 and 456 show the ESLint error Environment key jest/globals is unknown. The deployment fix removed react-app/jest from package.json eslintConfig.extends while retaining react-app. Screenshots 458 and 459 show compilation completing with warnings and the image being built. This changes test-specific lint configuration; it is not evidence that automated tests passed.

Warnings and worker restart

The successful frontend build retains a React Hook dependency warning involving apiUrl. A follow-up improvement is to include apiUrl in the useEffect dependency list and rebuild. Dependency deprecation notices also appear during npm installation.

Screenshot 475 contains a Gunicorn worker SIGKILL message with the diagnostic suggestion Perhaps out of memory, followed by a replacement worker boot and successful production requests. The screenshot does not prove an out-of-memory cause. Inspect Docker memory usage and full logs if this recurs; consider fewer Gunicorn workers on a constrained machine. The evidence supports a recovered functional application, not an error-free log history.

The development server warning is expected for the Flask development service. A favicon 404 is also visible. Successful API responses and database pings establish functional connectivity at the captured time.

Submission and demonstration

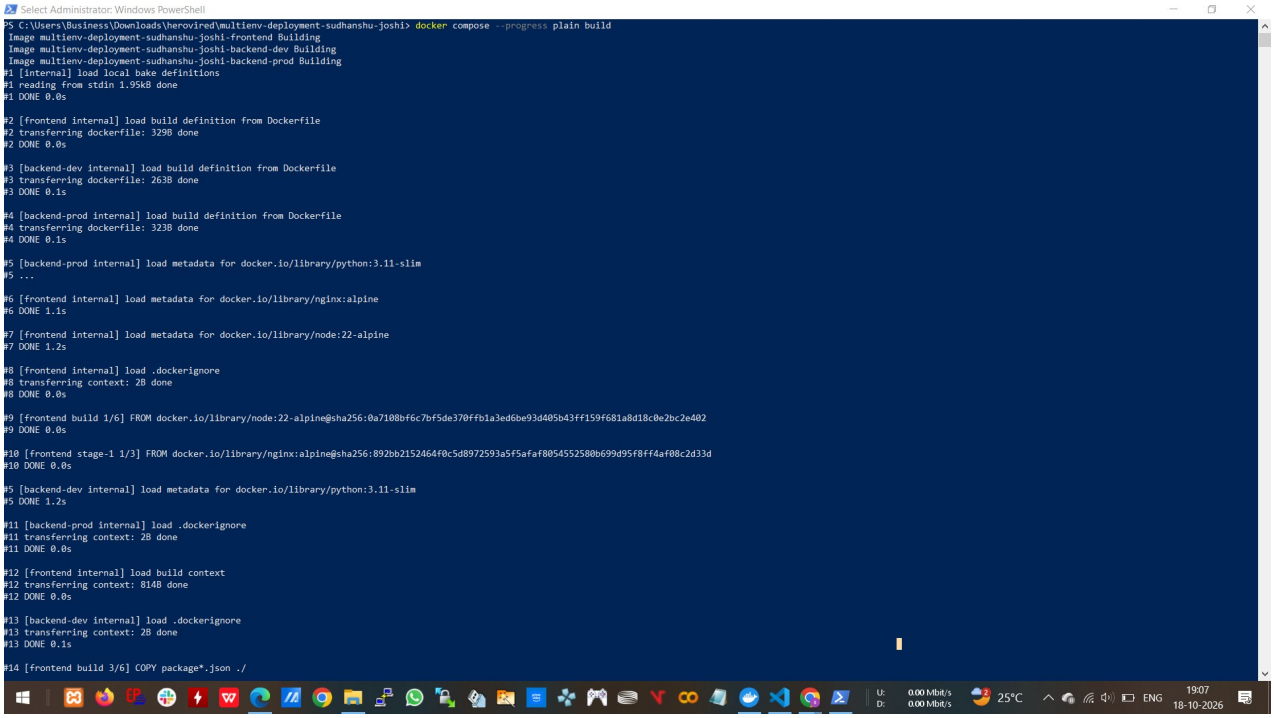
Repository: https://github.com/sjoshigit/multienv-deployment-sudhanshu-joshi
Submission file: submission/submission-info.txt
Student: Sudhanshu Joshi
Date: 18-10-2026
Deployment status: Working local deployment

Before submitting, include Docker configuration files, safe environment examples, this report and the screenshots in the repository. Confirm that the repository is public and upload submission-info.txt through the instructor portal. Do not commit private credentials. These publication steps are not verified by the supplied images.

For VIVA, demonstrate container status, create a ticket in each environment, show both React pages and database pings, and explain service-name DNS, port mappings, the named volume, the multi-stage frontend build and the distinction between Flask development serving and Gunicorn.

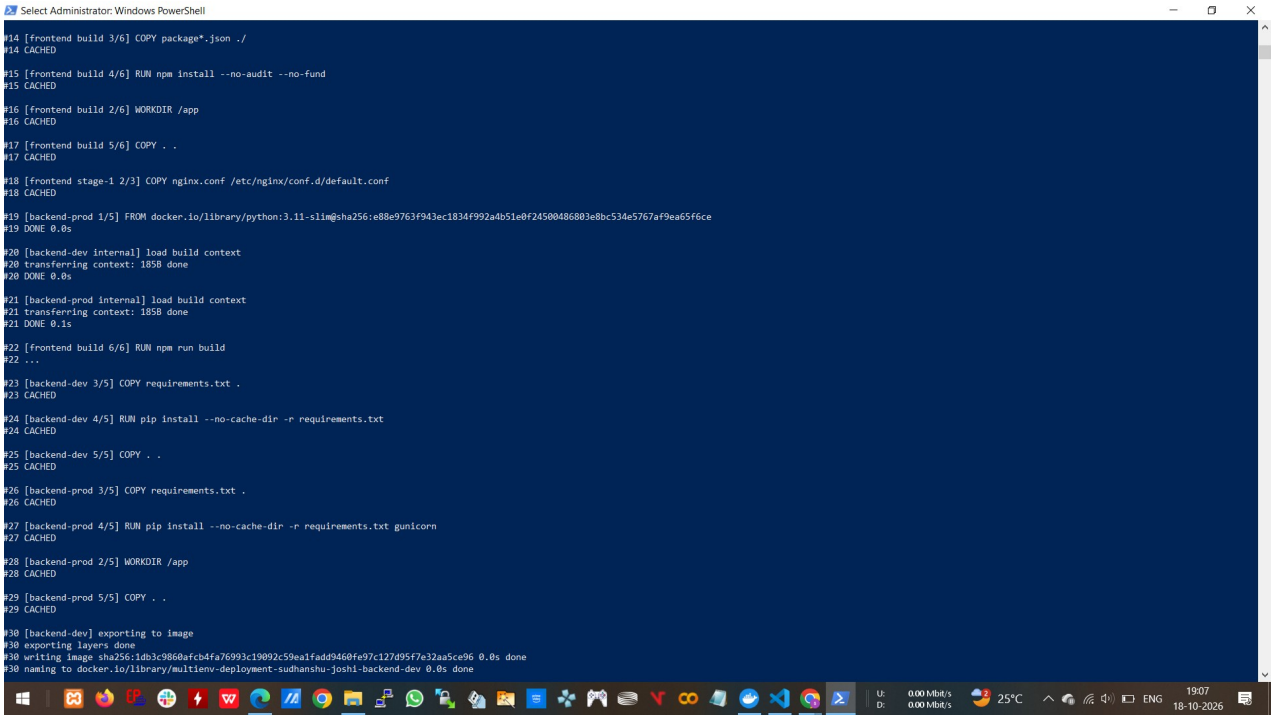
Screenshot evidence

Figure 1 Initial image build



Source: Screenshot (453).png. Compose loads Dockerfiles, image metadata and build contexts.

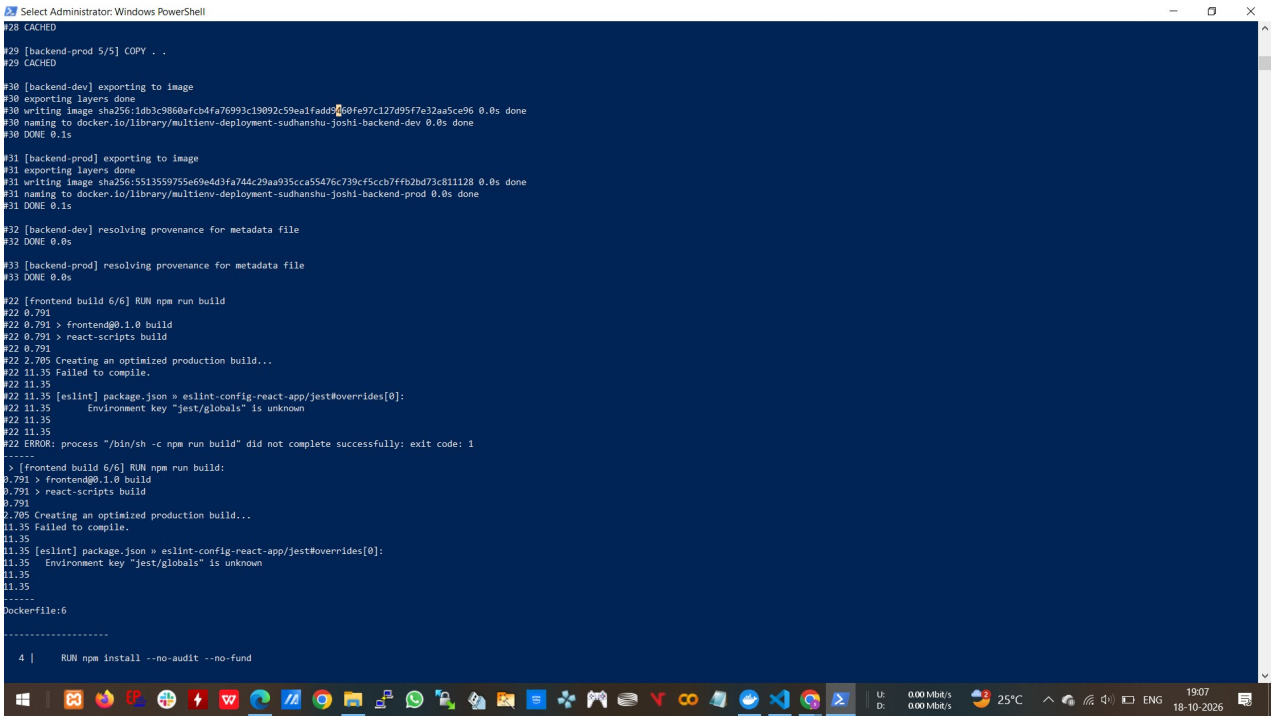
Figure 2 Backend build layers



Source: Screenshot (454).png. Dependency installation, cached layers and backend image export are visible.

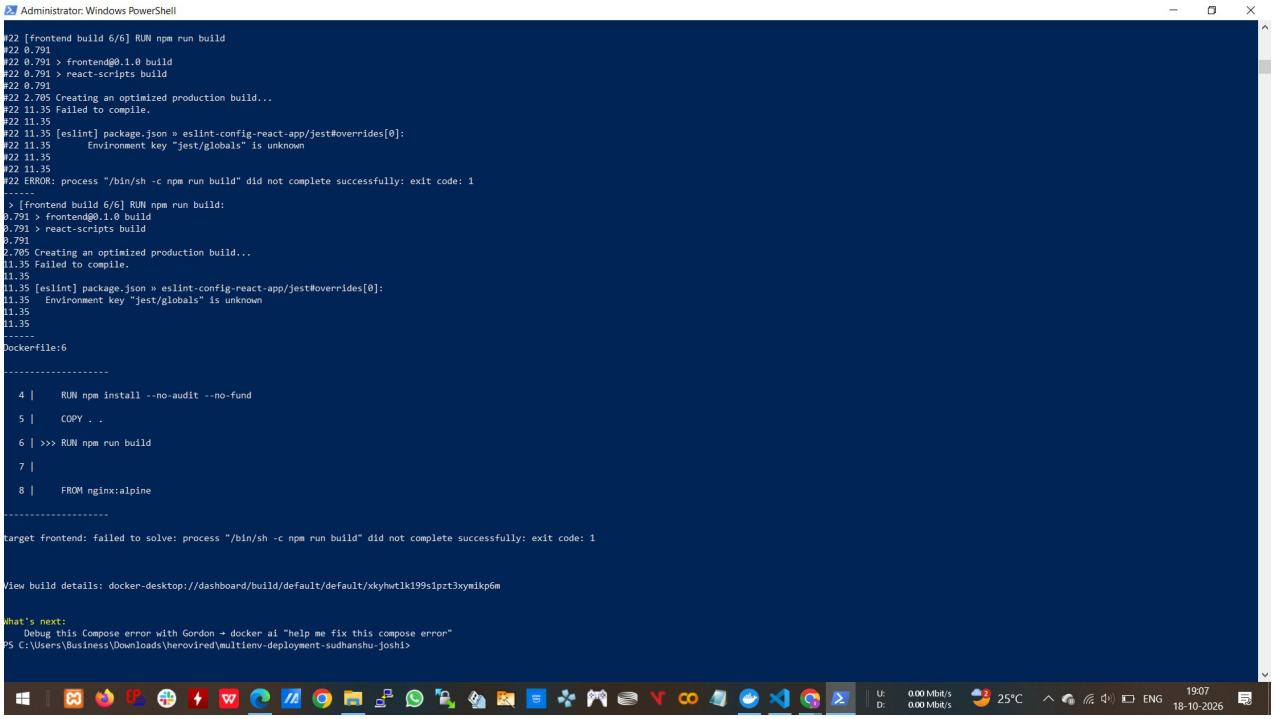
Screenshot evidence continued

Figure 3 Frontend compilation failure



Source: Screenshot (455).png. The build fails on the Jest ESLint environment configuration.

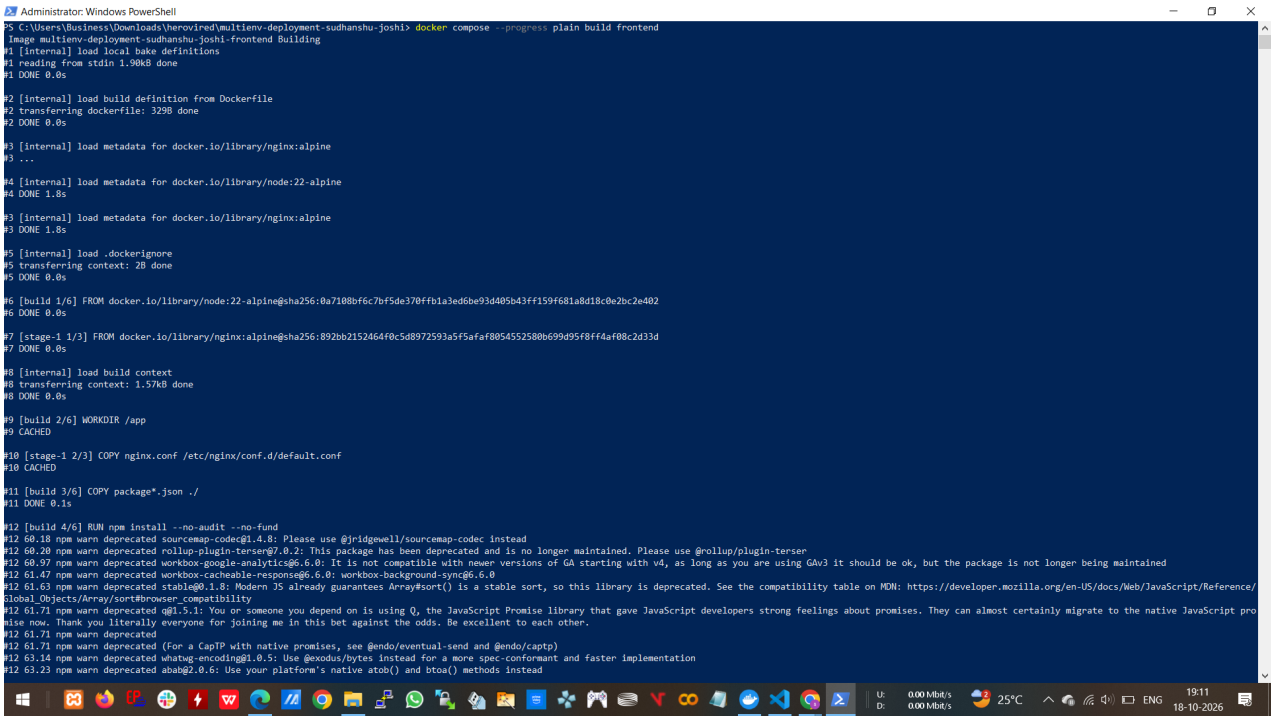
Figure 4 Failure details



Source: Screenshot (456).png. The frontend npm run build step exits with an error before the fix.

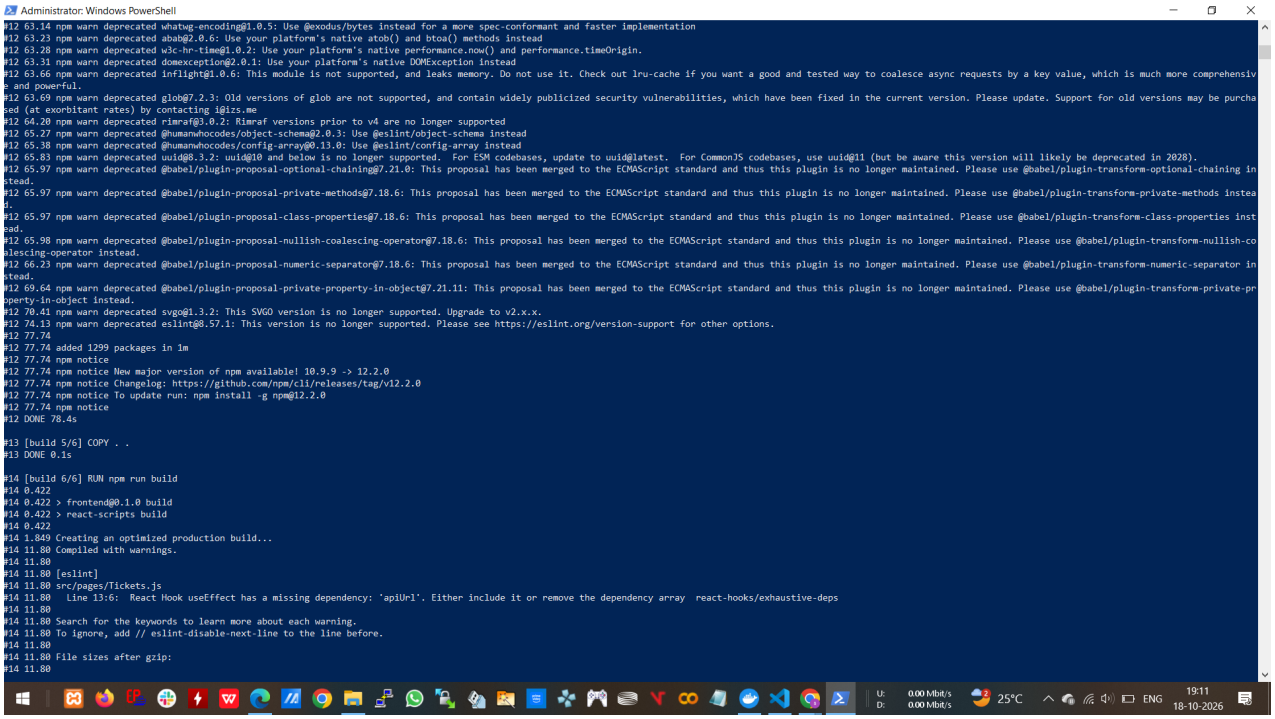
Screenshot evidence continued

Figure 5 Frontend rebuild



Source: Screenshot (457).png. A new frontend build installs Node dependencies after configuration changes.

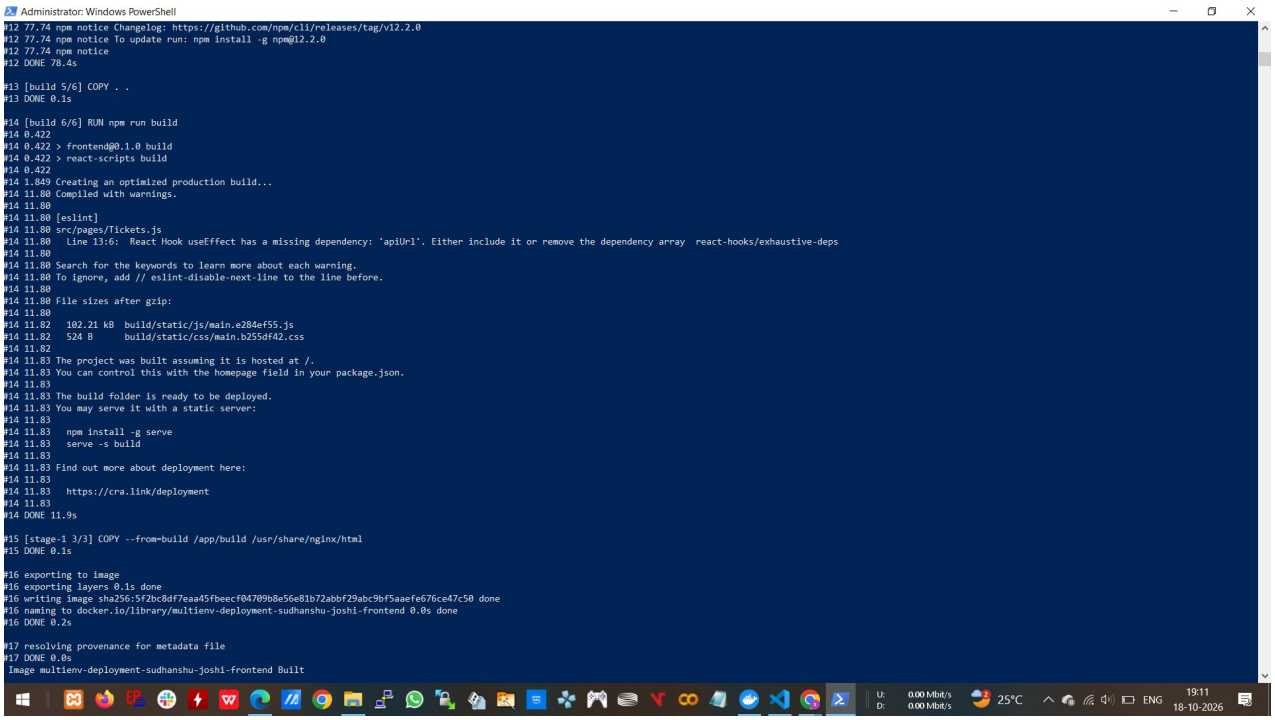
Figure 6 Compilation with warnings



Source: Screenshot (458).png. React compilation completes with warnings, including the apiUrl dependency warning.

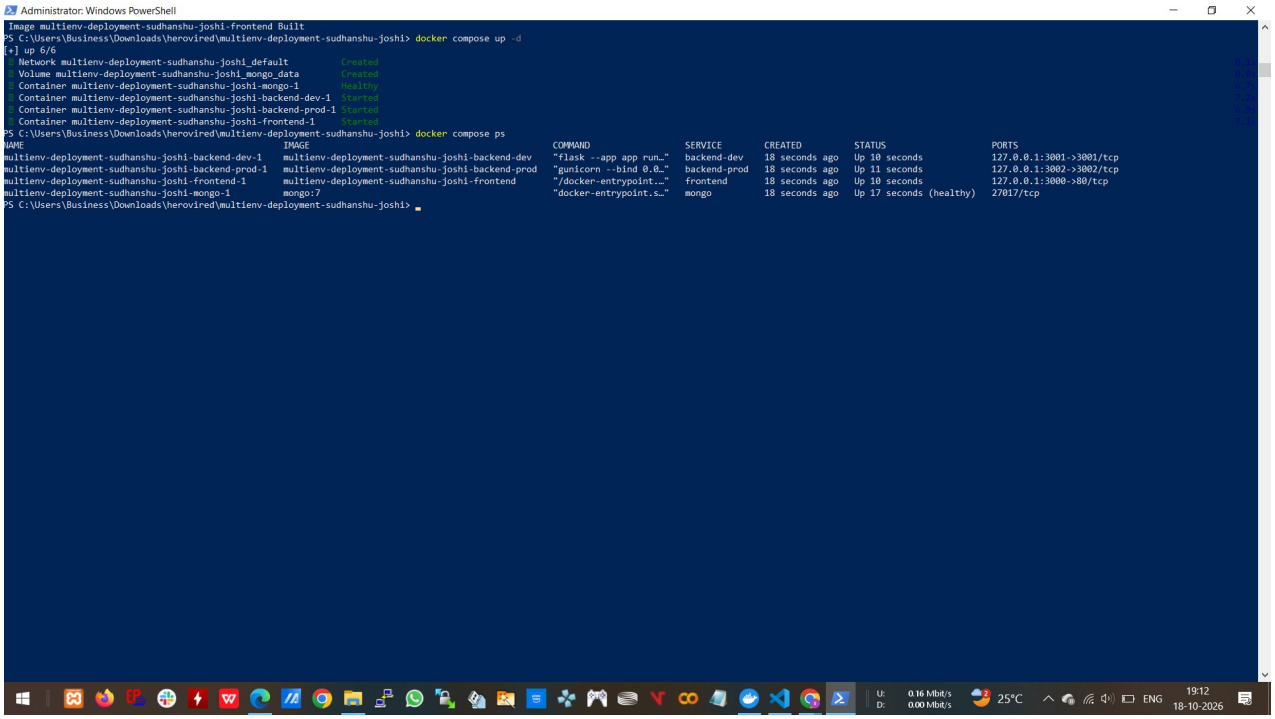
Screenshot evidence continued

Figure 7 Frontend image completed



Source: Screenshot (459).png. The static build is copied into Nginx and the frontend image is built.

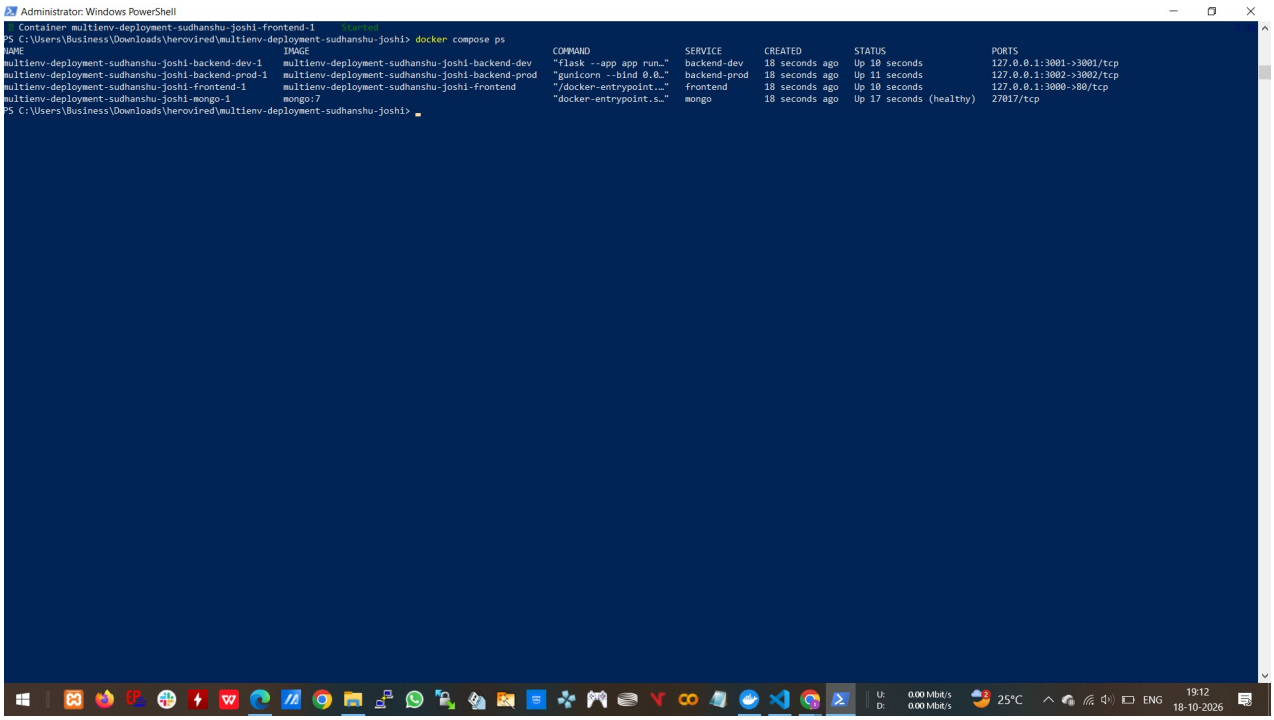
Figure 8 Services started



Source: Screenshot (460).png. Compose creates the network and volume; all four services run and MongoDB is healthy.

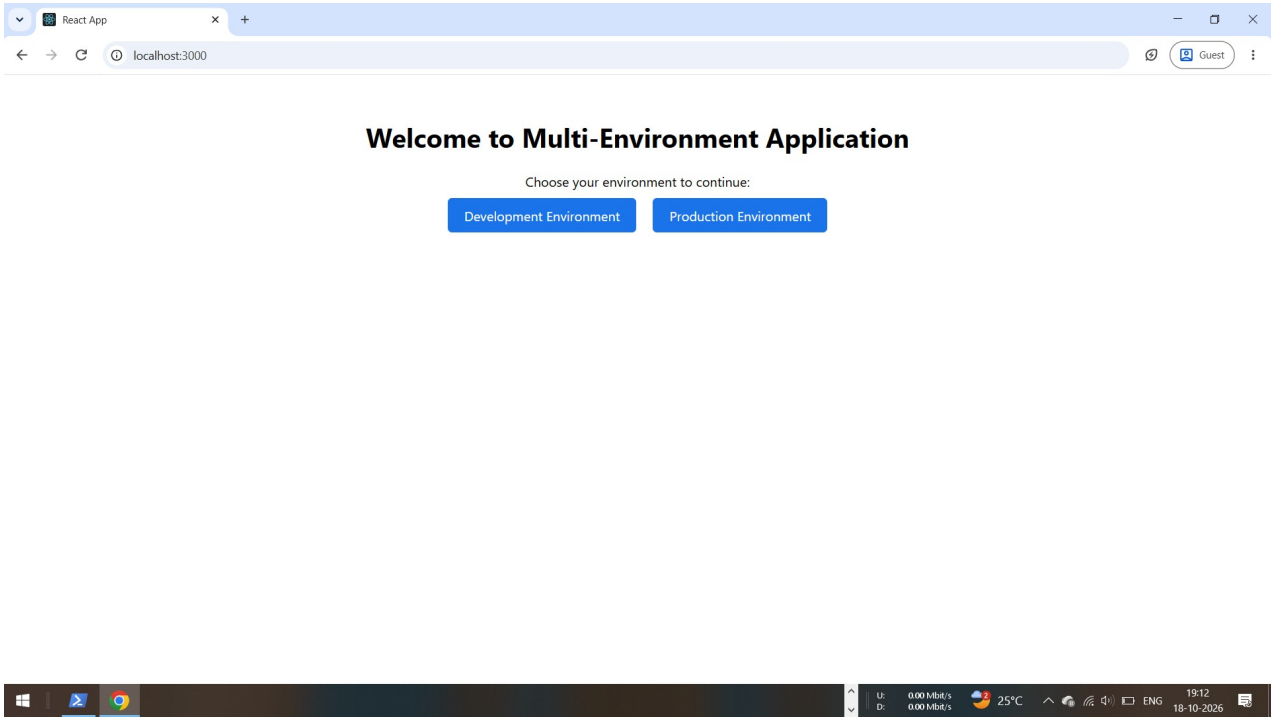
Screenshot evidence continued

Figure 9 Container status confirmation



Source: Screenshot (461).png. A repeated Compose status check lists all four running services.

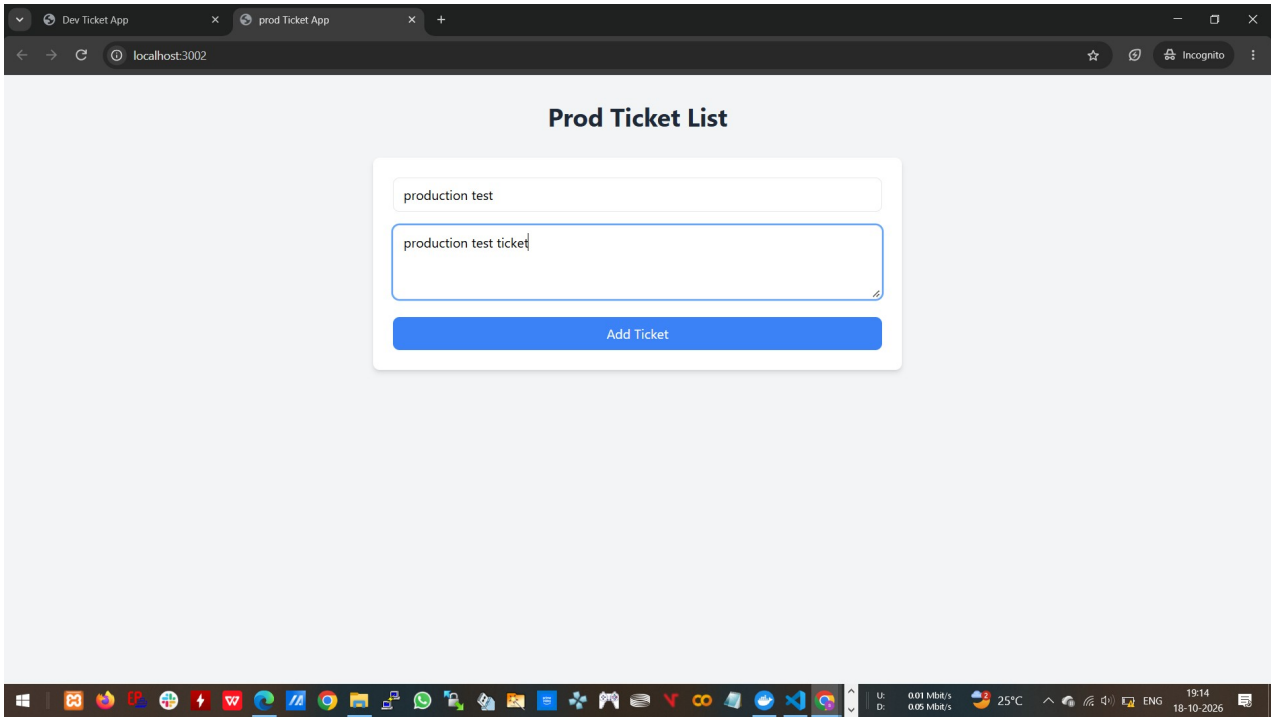
Figure 10 Frontend dashboard



Source: Screenshot (462).png. The dashboard at localhost:3000 offers development and production navigation.

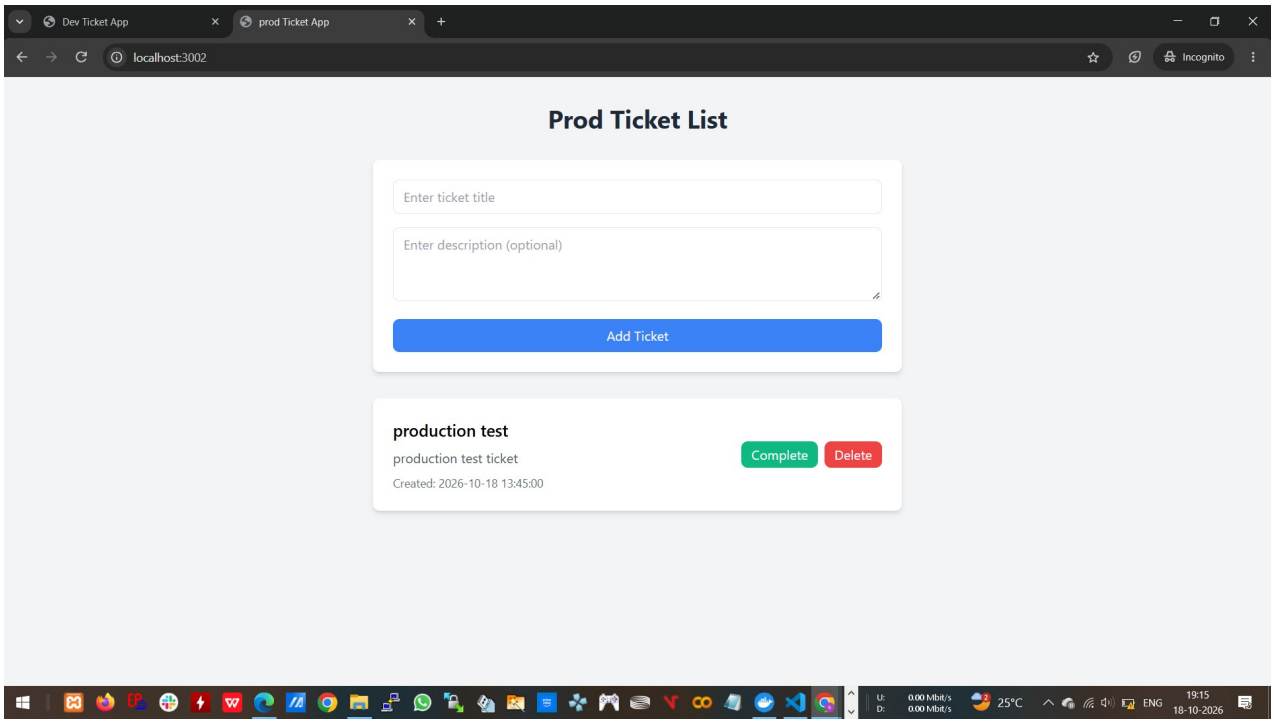
Screenshot evidence continued

Figure 11 Production ticket input



Source: Screenshot (467).png. The production management page contains a test ticket ready for submission.

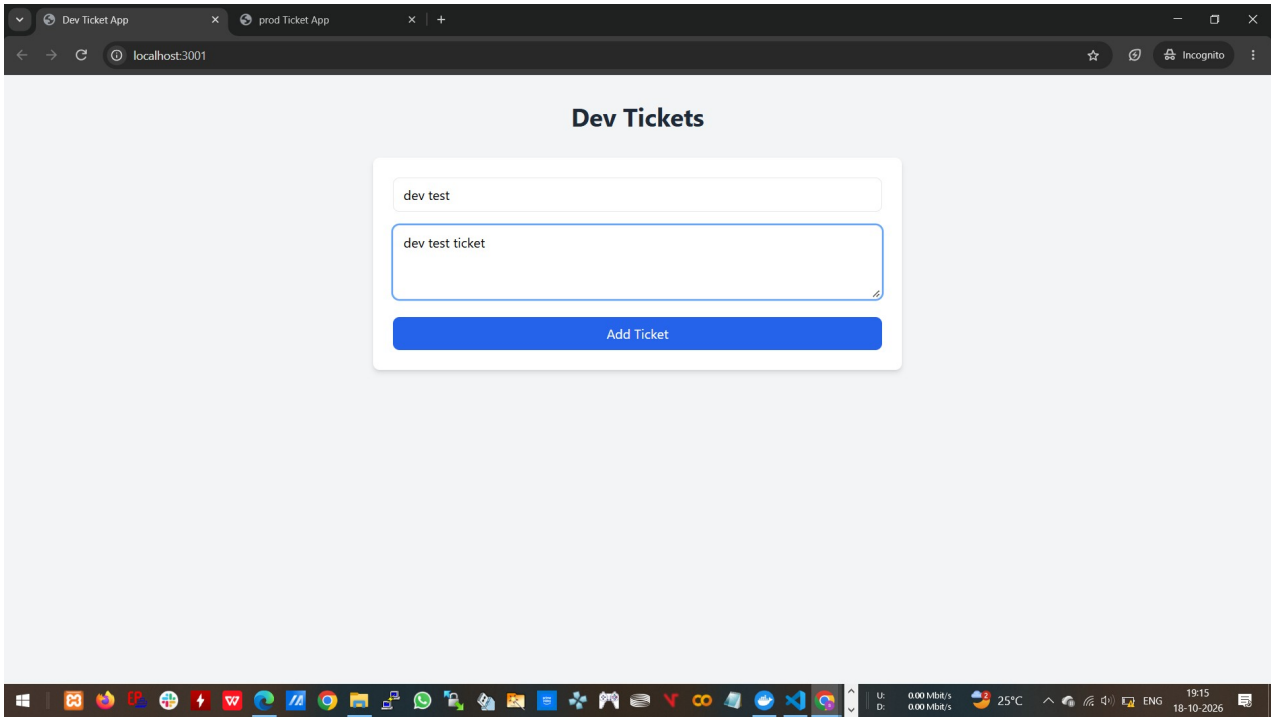
Figure 12 Production ticket stored



Source: Screenshot (468).png. The production ticket appears after submission.

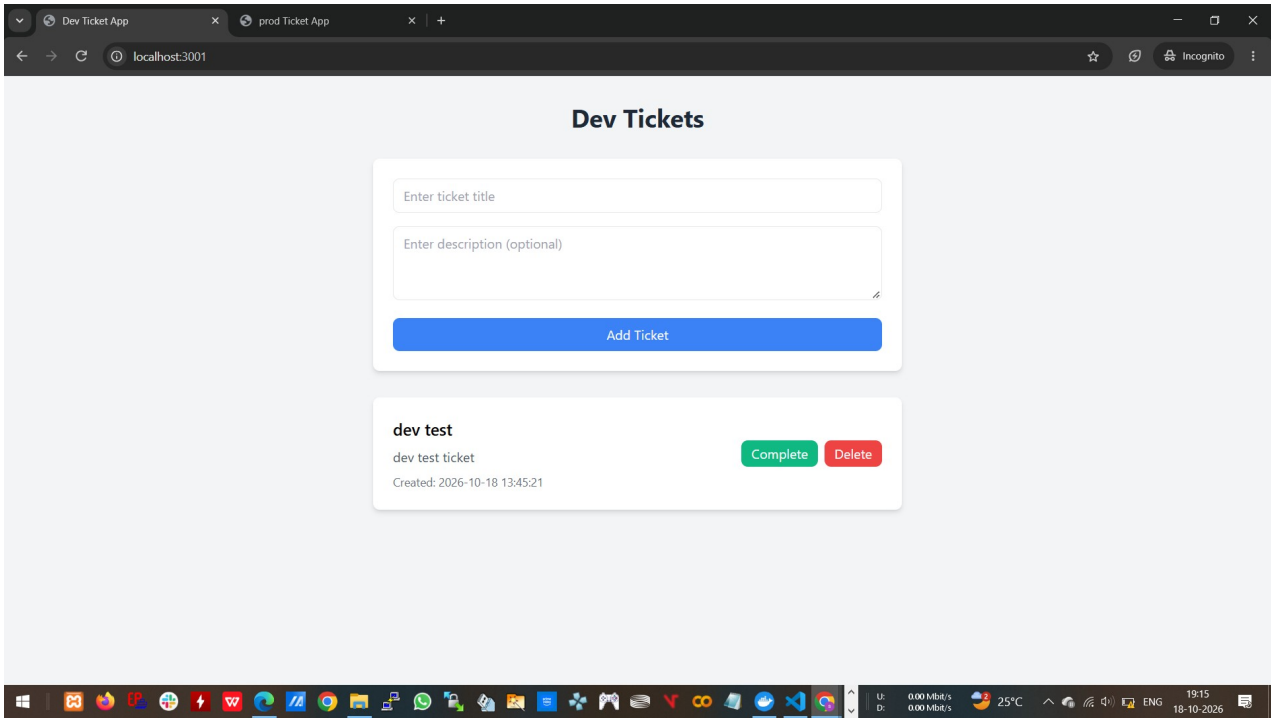
Screenshot evidence continued

Figure 13 Development ticket input



Source: Screenshot (469).png. The development management page contains a separate test ticket.

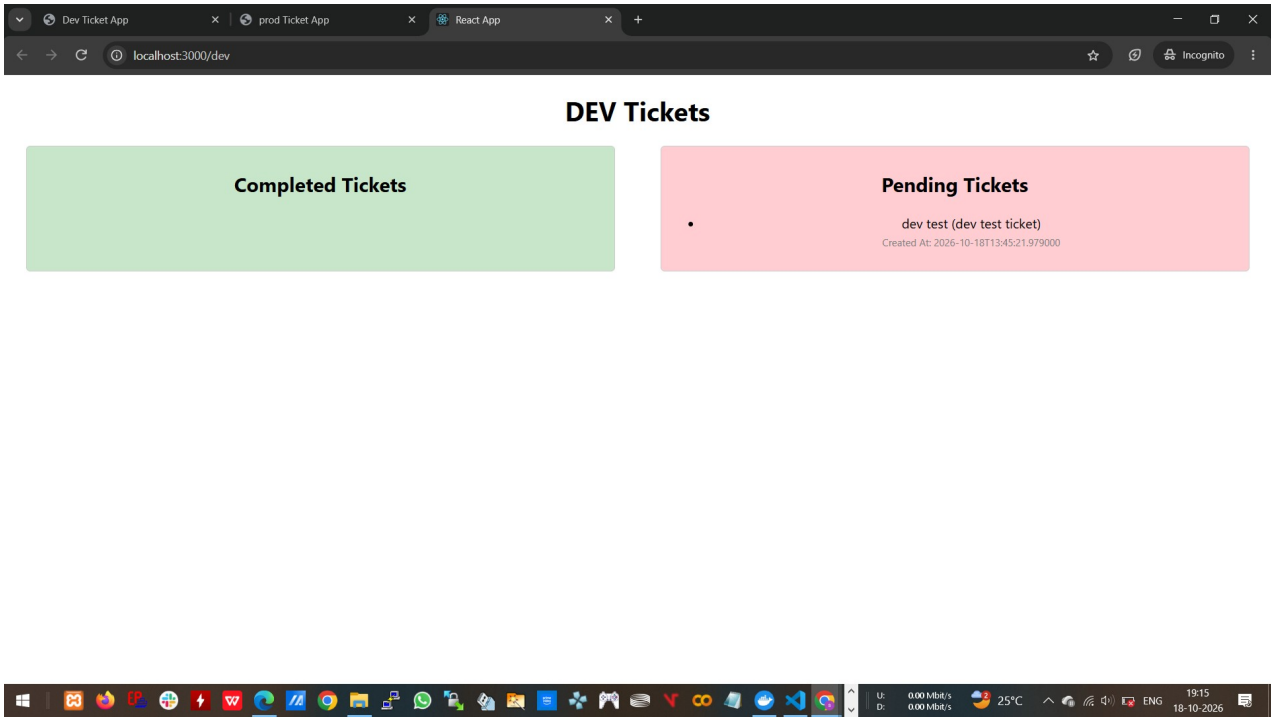
Figure 14 Development ticket stored



Source: Screenshot (470).png. The development ticket appears after submission.

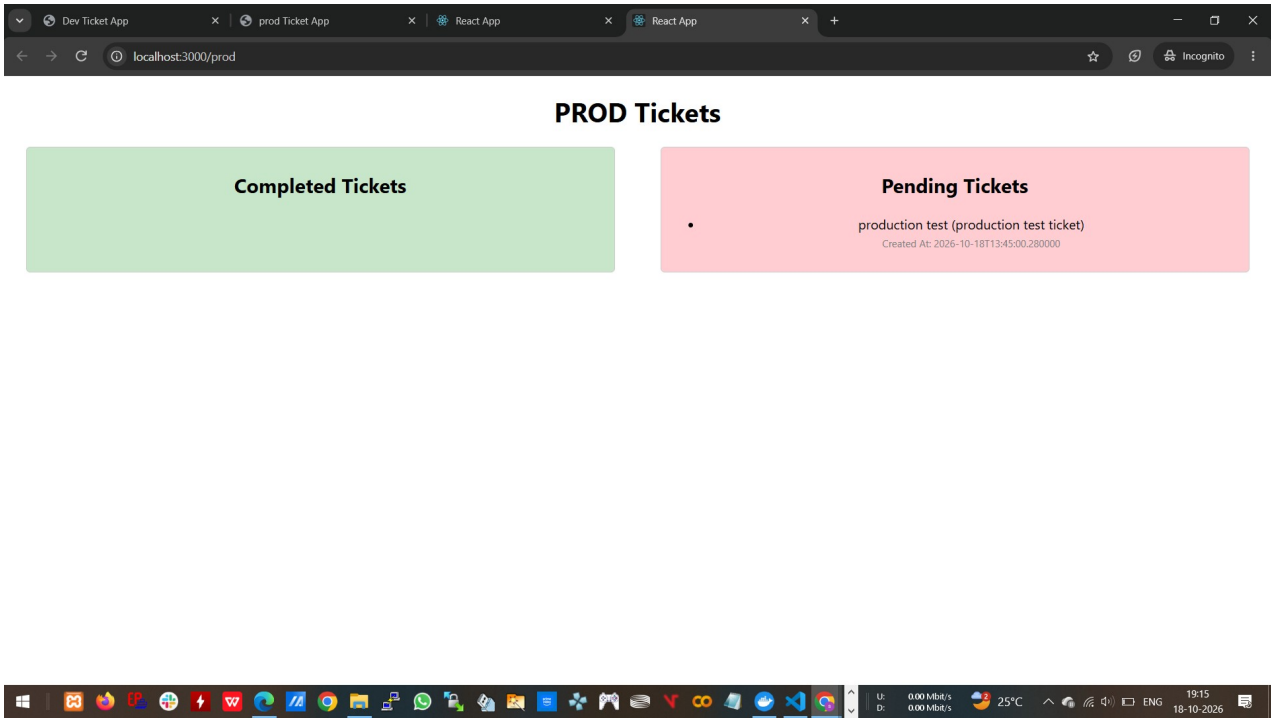
Screenshot evidence continued

Figure 15 Development React integration



Source: Screenshot (471).png. The React /dev page displays the development ticket in Pending Tickets.

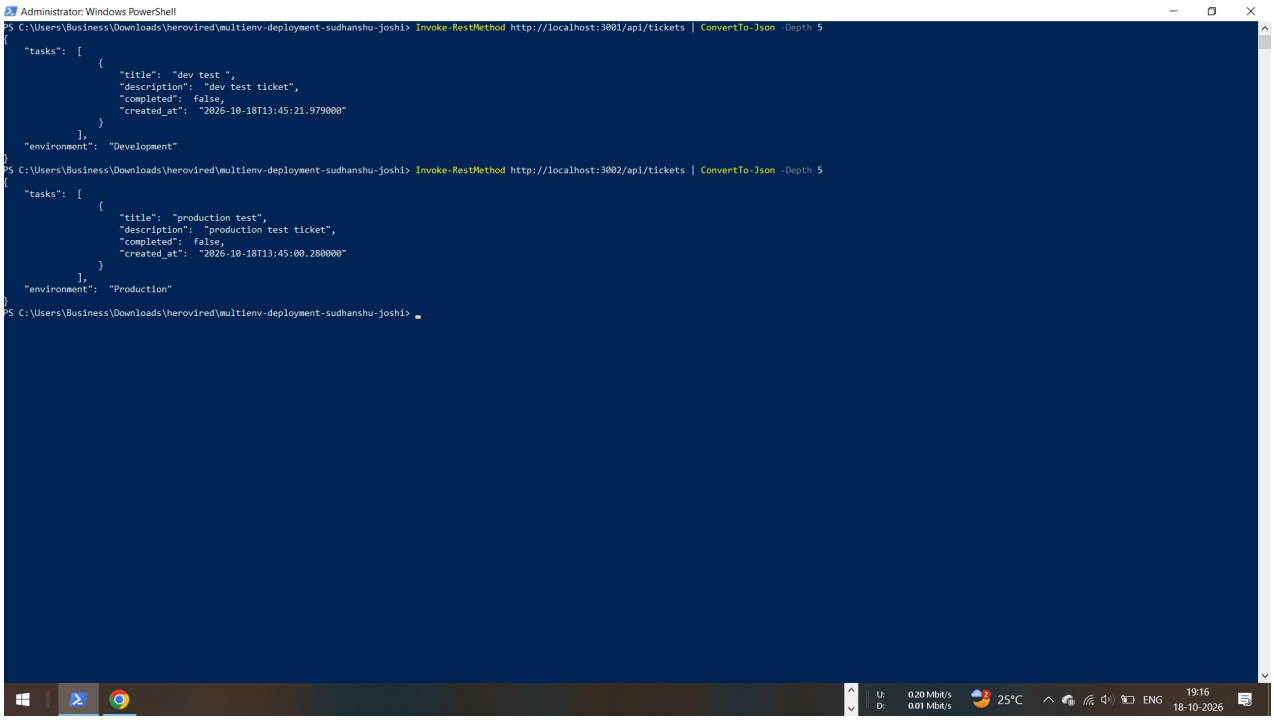
Figure 16 Production React integration



Source: Screenshot (472).png. The React /prod page displays the production ticket in Pending Tickets.

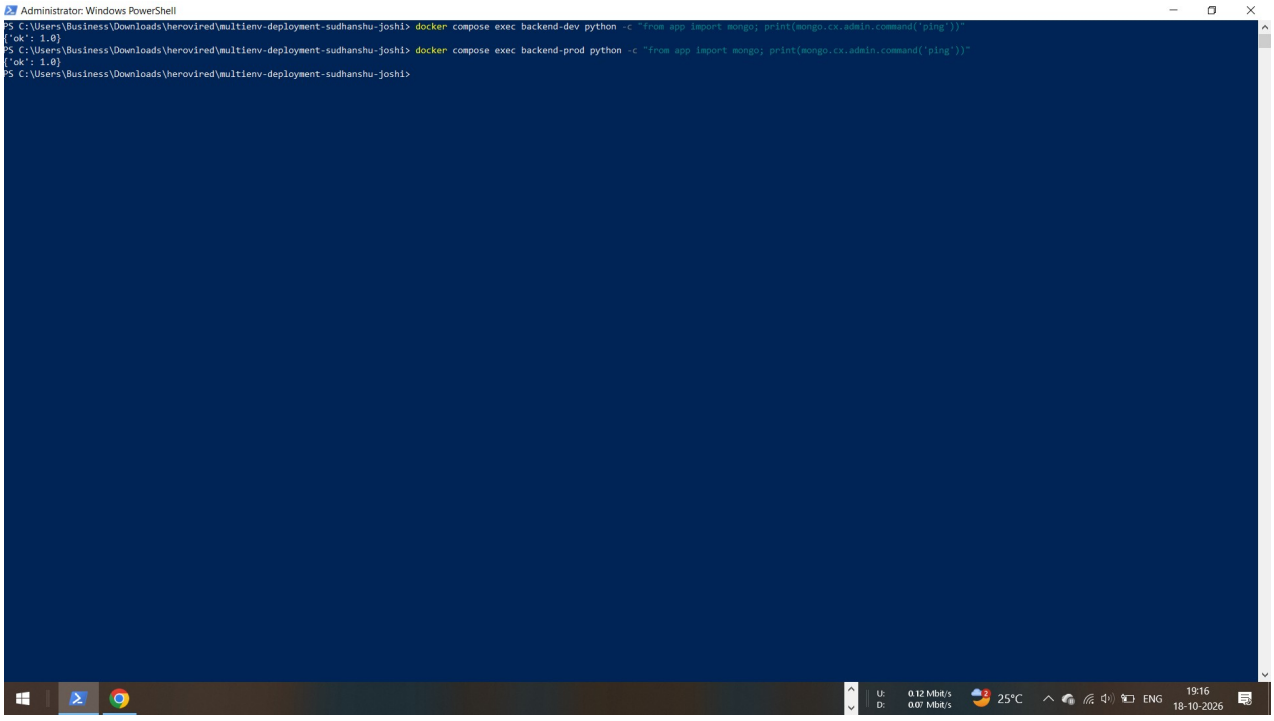
Screenshot evidence continued

Figure 17 API response verification



Source: Screenshot (473).png. PowerShell receives separate development and production JSON responses.

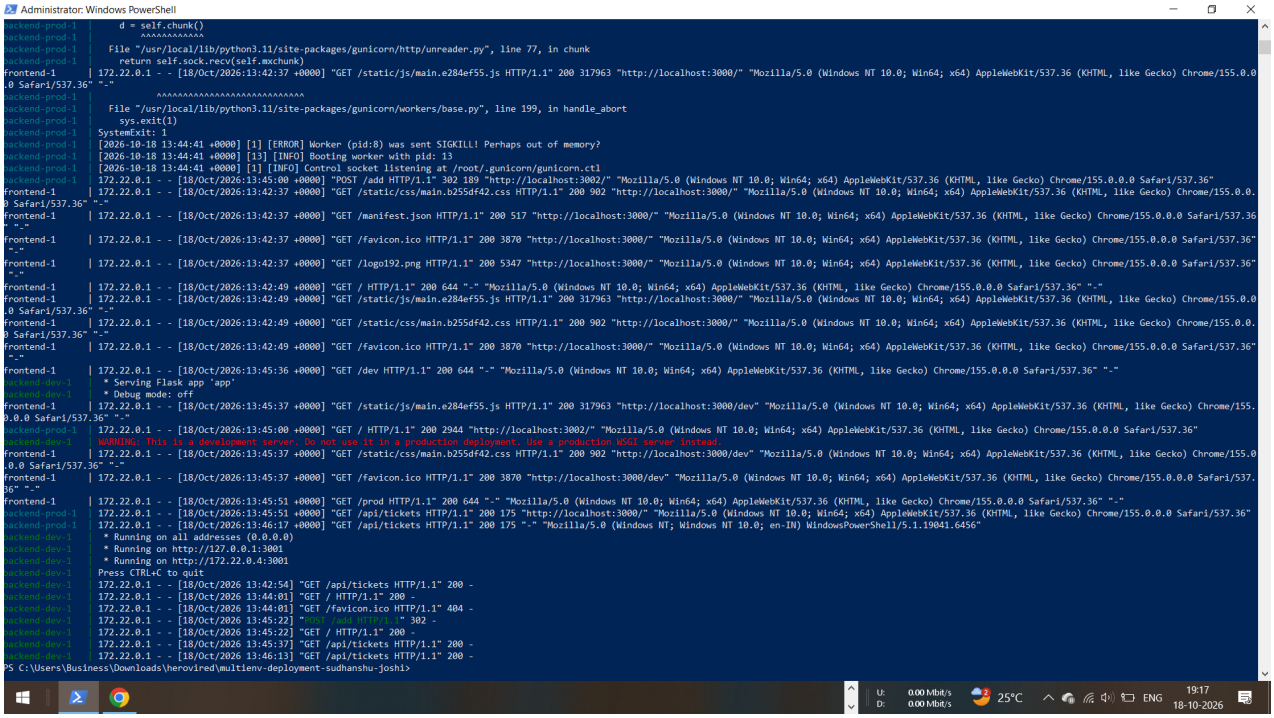
Figure 18 Database connectivity verification



Source: Screenshot (474).png. Both backend containers return ok 1.0 from MongoDB ping.

Screenshot evidence continued

Figure 19 Service logs and recovery



Source: Screenshot (475).png. HTTP successes appear alongside a Gunicorn worker termination and restart; see troubleshooting.

Infrastructure specifications and evidence

The additional screenshots document the local host, Docker installation and Compose network. They complete the infrastructure evidence for the local deployment option; AWS security groups are not applicable.

Operating system: Windows 10 Enterprise Evaluation, 10.0.19045, 64-bit

Processor: 11th Gen Intel Core i5-11400H at 2.70 GHz

CPU topology: 6 cores and 12 logical processors

Memory: 23.74 GiB reported by TotalPhysicalMemory / 1GB

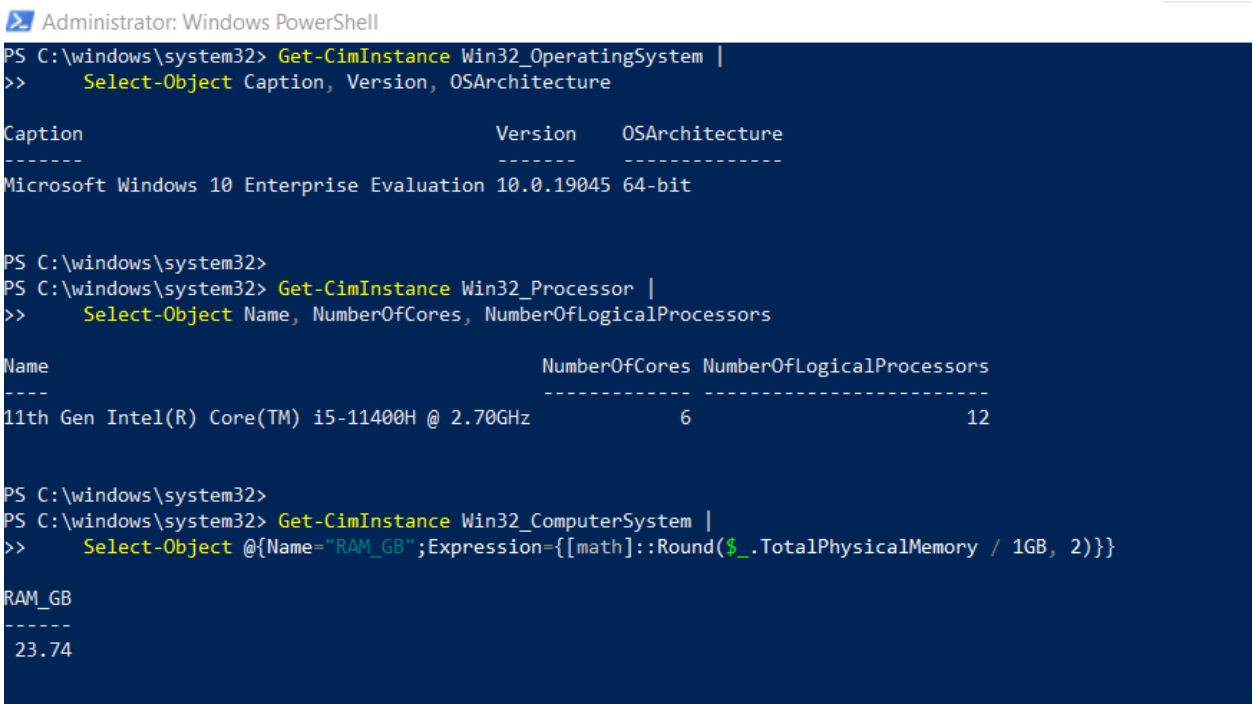
Docker Desktop: 4.95.0 (242831)

Docker client and engine: 29.8.2; API 1.56; context desktop-linux

Container platform: linux/amd64

Docker Compose: v5.6.0

Figure 20 Windows host specifications

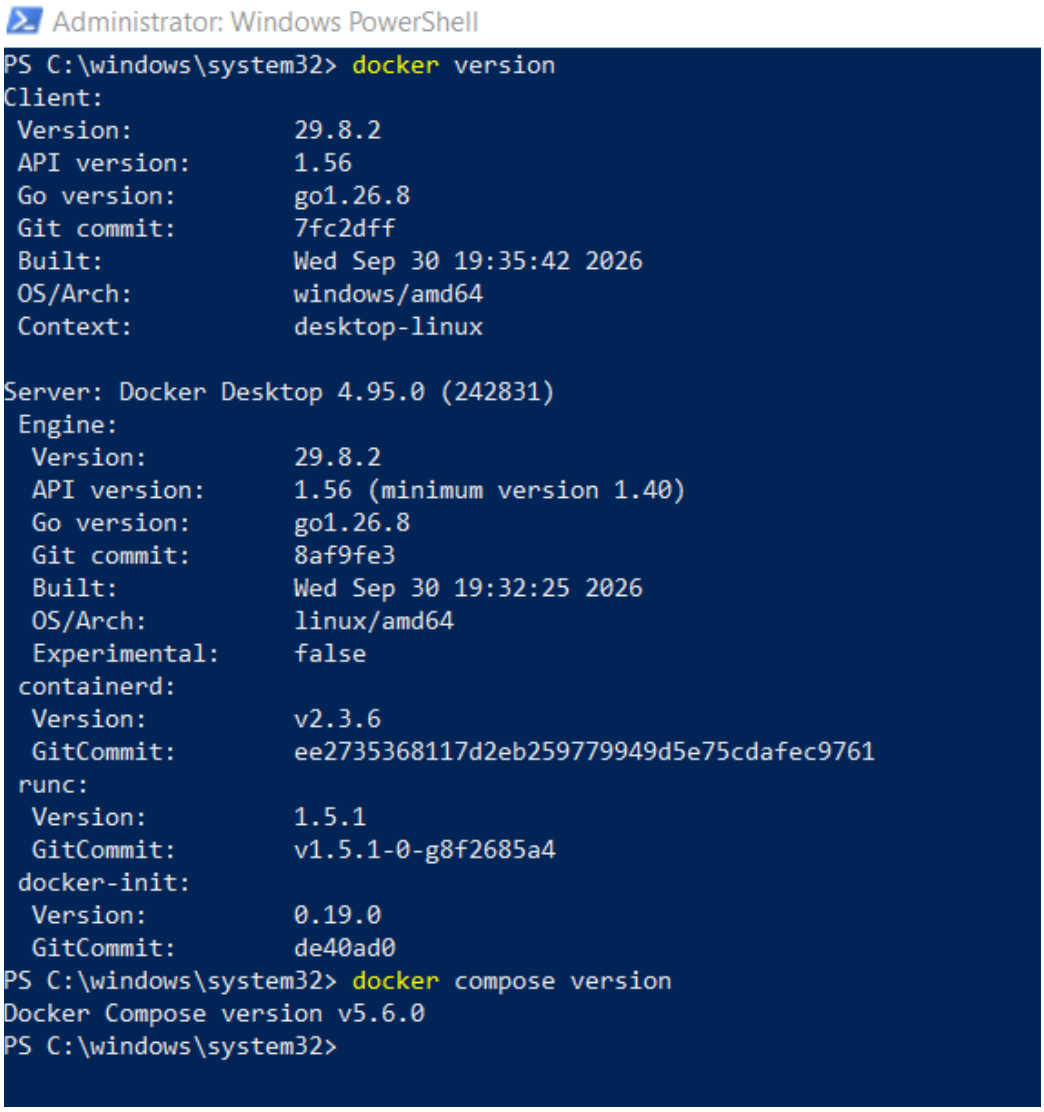


Source: image(2).png. PowerShell identifies the operating system, processor, CPU topology and host memory.

Docker installation evidence

The version output shows both Client and Server sections, confirming connectivity to the Docker Desktop Linux engine. The client runs on windows/amd64 and the server on linux/amd64. Docker Compose is installed and reports v5.6.0.

Figure 21 Docker and Compose versions

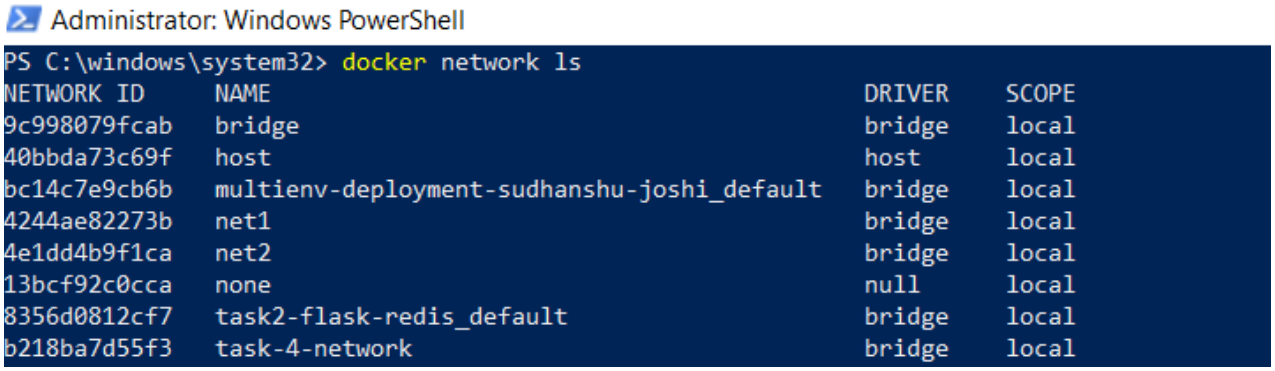


Source: image(3).png. Docker Desktop, engine, client API and Compose versions are visible.

Compose network configuration evidence

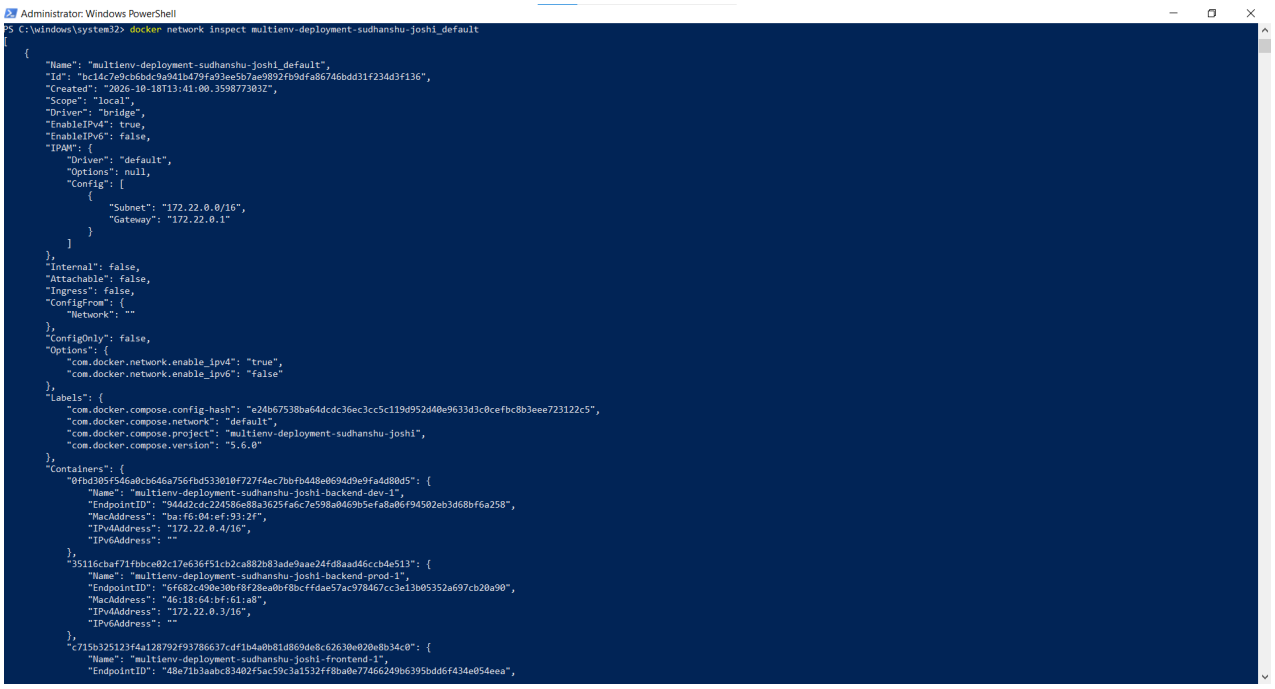
The deployment uses the user-defined bridge network multienv-deployment-sudhanshu-joshi_default. Network inspection shows local scope, IPv4 enabled, subnet 172.22.0.0/16 and gateway 172.22.0.1. These are Docker network addresses, not public access URLs.

Figure 22 Network inventory



Source: image(4).png. The project default network appears with bridge driver and local scope.

Figure 23 Network inspection



Source: image(5).png. The network name, driver, subnet, gateway and Compose labels are shown.

Container network membership evidence

All four application services are attached to the same Compose network. The addresses below describe the captured deployment and may change when containers are recreated. Application configuration uses service-name DNS rather than these transient addresses.

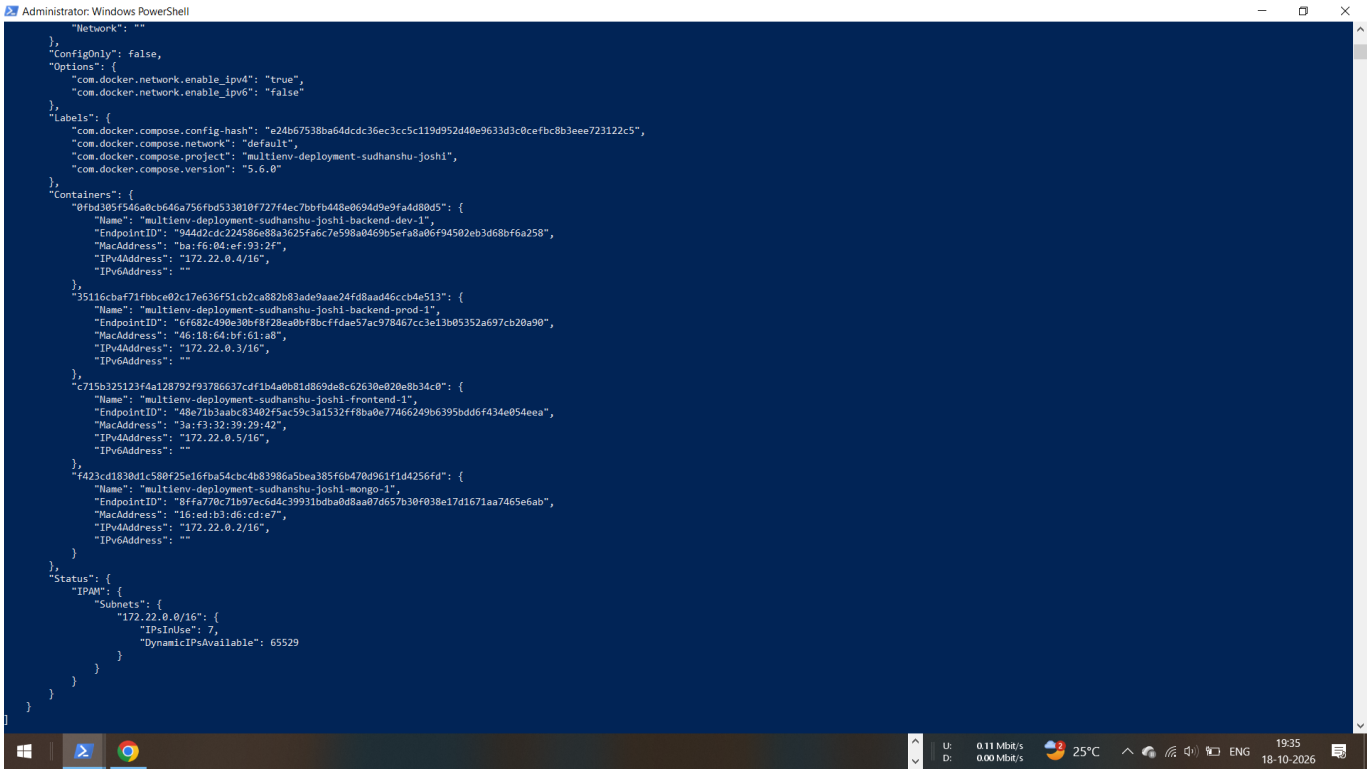
mongo — 172.22.0.2

backend-prod — 172.22.0.3

backend-dev — 172.22.0.4

frontend — 172.22.0.5

Figure 24 Attached services and addresses



Source: image(6).png. Network inspection lists development, production, frontend and MongoDB container endpoints.

Infrastructure coverage: local host specifications are documented in Figure 20; Docker readiness in Figure 21; network creation and configuration in Figures 22 to 24. Existing Figures 8 and 9 show host port bindings, and Figure 18 demonstrates successful database connectivity from both backends. No EC2 instance or AWS security group was required for this local deployment.